Système d'exploitation
CM2

Pierre Rousselin
rousselin@univ-paris13.fr

Université Sorbonne Paris Nord
L2 informatique et double-licence

février 2026

Statut de sortie et message d'erreur

Rappels de C : pointeurs (1)

Rappels de C, encore : fonctions avec paramètres pointeurs

Signaux : une introduction

Attendre : `wait`

Exécuter un programme : `exec`

Zombies

Mort d'un processus

- `return n;` depuis `main` ;
- `exit(n);` depuis n'importe où.

Dans les deux cas, appel système `exit` pour mettre fin au processus. L'entier `n` donné est *le statut de sortie* du processus, un petit entier. En général, on considère que

- 0 signale que tout s'est bien passé ;
- > 0 signale qu'une erreur s'est produite.

Toutes les ressources associées au processus sont alors libérées par le système.

Mort d'un processus

- `return n;` depuis `main` ;
- `exit(n);` depuis n'importe où.

Dans les deux cas, appel système `exit` pour mettre fin au processus. L'entier `n` donné est *le statut de sortie* du processus, un petit entier. En général, on considère que

- 0 signale que tout s'est bien passé ;
- > 0 signale qu'une erreur s'est produite.

Toutes les ressources associées au processus sont alors libérées par le système. Un processus peut également mourir de façon moins naturelle en recevant un signal, comme dans la *fork bomb* précédente, où l'ancêtre de la myriade de processus tue tous ses descendants avec

```
kill(0, SIGTERM);
```

(Plus d'informations sur les signaux dans un autre cours).

errno et perror

On a utilisé précédemment

```
case -1:
    perror("fork");
```

Et cela a produit l'affichage :

```
fork: Resource temporarily unavailable
```

- Rien de magique là-dedans.
- Chaque processus (en fait chaque *thread*) possède une variable globale entière `errno`.
- Les appels systèmes (ou d'autre fonctions de la bibliothèque standard), lorsqu'ils échouent, mettent dans `errno` une valeur entière > 0 qui donnent des renseignements sur l'erreur.
- La fonction

  ```
  #include <string.h>
  char *strerror(int errnum);
  ```

  permet de traduire ce numéro en une chaîne compréhensible.
- La fonction

  ```
  #include <stdio.h>
  void perror(const char *s);
  ```

  affiche la chaîne d'adresse `s`, suivi du message correspondant au `errno` courant et d'un caractère de fin de ligne.

Les différents messages d'erreur

```c
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
int main()
{
    char errno_string[4];
    int i;
    for (i = 0; i < 156; ++i) {
        errno = i;
        sprintf(errno_string, "%3d", i);
        perror(errno_string);
    }
    return 0;
}
```

donne :

```
$ ./a.out
  0: Success
  1: Operation not permitted
  2: No such file or directory
  3: No such process
  (...)
```

erreur : réflexes

- (Presque) toujours tester la valeur de retour d'un appel système (en général -1 pour indiquer une erreur).
- En cas d'erreur, `perror` avec comme argument quelque chose qui aide à repérer où l'erreur s'est produite (par exemple le nom de l'appel système).
- Si c'est irrécupérable (ça l'est probablement), `exit` avec un argument non nul pour mettre fin au processus.
- Constructions fréquentes en C :

```
if ((pid = fork()) != -1)
    /* ... */
while ((n = read(...)) > 0)
    /* ... */
```

  Affectation et test en même temps, ok car la valeur d'une instruction d'affectation est la valeur affectée.

Statut de sortie et message d'erreur

Rappels de C : pointeurs (1)

Rappels de C, encore : fonctions avec paramètres pointeurs

Signaux : une introduction

Attendre : `wait`

Exécuter un programme : `exec`

Zombies

Mémoire

Modèle mental :

- La mémoire associée à un processus est comme un très grand tableau
- où chaque case peut contenir un octet d'informations
- et a un indice dans le tableau (octet numéro 0, octet numéro 1, ...).
- On dit que l'architecture de la machine est *byte-addressable*, c'est-à-dire qu'on peut accéder à chaque octet par son indice ou adresse.

Mémoire

Modèle mental :

- La mémoire associée à un processus est comme un très grand tableau
- où chaque case peut contenir un octet d'informations
- et a un indice dans le tableau (octet numéro 0, octet numéro 1, ...).
- On dit que l'architecture de la machine est *byte-addressable*, c'est-à-dire qu'on peut accéder à chaque octet par son indice ou adresse.
- Une *variable* est une suite continue de cases (octets) dans ce très grand tableau.
- Un *pointeur* est une variable un peu spéciale : elle est destinée à *contenir une adresse*.

Exemple 1 (inutile)

```c
#include <stdio.h>
int main(void)
{
    int i = 42, j;
    int *p;

    p = &i; /* mettre dans p l'adresse où commence i */
    /* afficher l'entier qui commence à l'adresse p */
    printf("*p = %d\n", *p);

    /* ajouter 3 à l'entier qui commence à l'adresse p */
    *p = *p + 3;
    printf("*p = %d\n", *p);
    printf("i = %d\n", i);

    /* mettre dans p l'adresse où commence j. */
    p = &j;
    *p = 1337;
    printf("*p = %d\n", *p);
    printf("j = %d\n", j);
    return 0;
}
```

Pointeurs et opérateurs

- *Déclaration* d'une variable `id` pointeur sur une donnée de type `type`.

  `type *id;`
- ... ou `type* id;` ou `type * id;` selon les goûts.
- *Déréférencement* (accès à la valeur contenue à cette adresse) d'un pointeur `id`, de type `type *` : `*id` (de type `type`).
- *Adresse* d'une variable `var` de type `type` : `&var` (de type `type *`).

Simuler un passage par référence

```c
#include <stdio.h>
void swap(int *n, int *m);
int main(void)
{
    int n = 3, m = 4;
    swap(&n, &m);
    printf("n = %d, m = %d\n", n, m);
    return 0;
}
void swap(int *n, int *m)
{
    int tmp = *n;
    *n = *m;
    *m = tmp;
}
```

Peut économiser des copies coûteuses

```
struct grosse_struct {
    int gros_tab[1000];
    double autre_gros_tab[200];
    /* ... */
};
```

Au lieu de :

```
void f(struct grosse_struct gs)
{
    printf("%d\n", gs.gros_tab[999]);
}
```

Peut économiser des copies coûteuses

```
struct grosse_struct {
    int gros_tab[1000];
    double autre_gros_tab[200];
    /* ... */
};
```

Au lieu de :

```
void f(struct grosse_struct gs)
{
    printf("%d\n", gs.gros_tab[999]);
}
```

On écrit plutôt :

```
void f(struct grosse_struct *gs)
{
    printf("%d\n", gs->gros_tab[999]);
}
```

Peut économiser des copies coûteuses

```
struct grosse_struct {
    int gros_tab[1000];
    double autre_gros_tab[200];
    /* ... */
};
```

Au lieu de :

```
void f(struct grosse_struct gs)
{
    printf("%d\n", gs.gros_tab[999]);
}
```

On écrit plutôt :

```
void f(struct grosse_struct *gs)
{
    printf("%d\n", gs->gros_tab[999]);
}
```

(taille d'un pointeur sur la plupart des machines d'aujourd'hui : 8 octets).

Statut de sortie et message d'erreur

Rappels de C : pointeurs (1)

Rappels de C, encore : fonctions avec paramètres pointeurs

Signaux : une introduction

Attendre : `wait`

Exécuter un programme : `exec`

Zombies

Exemple de L1

```
int main(void) {
    int n;
    scanf("%d", &n);
}
```

- Lire sur le terminal une chaîne de caractères, interpréter cette chaîne comme l'écriture en décimal d'un entier signé, et mettre le résultat à l'adresse de `n`.
- La fonction `scanf` utilise l'adresse donnée pour savoir où écrire (et le spécificateur de format, ici `%d` pour savoir combien d'octets écrire).

Exemples et contre-exemples

```
int main(void) {
    int *n;
    scanf("%d", n);
}
```

Exemples et contre-exemples

```
int main(void) {
    int *n;
    scanf("%d", n);
}
```

Erreur à l'exécution : l'adresse n'est pas initialisée, on ne sait même pas où `scanf` va écrire !

Exemples et contre-exemples

```
int main(void) {
    int *n = malloc(sizeof(int));
    scanf("%d", n);
}
```

Exemples et contre-exemples

```
int main(void) {
    int *n = malloc(sizeof(int));
    scanf("%d", n);
}
```

Ok, mais dans ce cas

- tester la valeur de retour de `malloc` est recommandé ;
- il faudrait `free` la mémoire d'adresse `n` à un moment.

Si `n` n'est utilisée que localement, aucun intérêt par rapport à l'utilisation d'une variable de la pile.

Exemples et contre-exemples

```
int n; /* globale */
int main(void) {
    scanf("%d", &n);
}
```

Exemples et contre-exemples

```
int n; /* globale */
int main(void) {
    scanf("%d", &n);
}
```

- Ok, variable globale, visible partout dans le fichier.
- En général ça va à l'encontre de certains principes de programmation (encapsulation), mais ça peut parfois simplifier bien des choses (utilisé en programmation système).

Arguments pointeurs, lecture et écriture

Beaucoup de fonctions ont des paramètres pointeurs mais avec des intentions différentes :

- certains arguments pointeurs sont utilisés seulement pour *lire* des données : dans ce cas, les types pointés sont précédés du mot-clé `const`.

```
#include <string.h>
char *strstr(const char *meule_de_foin, const char *aiguille);
```

- certains sont utilisés uniquement pour écrire des données : ici `dest` en écriture et `src` en lecture [1]

```
#include <string.h>
void *memcpy(void *restrict dest,
                const void *restrict src, size_t n);
```

- certains sont utilisés en lecture et écriture (plus rare), cas dans ce cours avec `addrlen` de l'appel système `accept`.

```
#include <sys/socket.h>
int accept(int sockfd, struct sockaddr *restrict addr,
                  socklen_t *restrict addrlen);
```

1. pour les plus curieux, `restrict` signifie qu'on promet au compilateur que les zones mémoires ne se chevaucheront pas, ce qui lui permet d'optimiser le code produit

Statut de sortie et message d'erreur

Rappels de C : pointeurs (1)

Rappels de C, encore : fonctions avec paramètres pointeurs

Signaux : une introduction

Attendre : `wait`

Exécuter un programme : `exec`

Zombies

Introduction

- Les signaux sont une forme assez rudimentaire mais indispensable de *communication inter-processus* (*interprocess-communication*, *IPC*).
- Un signal peut être envoyé à un processus par un autre processus (du même *user* propriétaire) ou bien par le noyau.
- Les signaux diffèrent selon les systèmes.

```
$ kill -l
 1) SIGHUP    2) SIGINT    3) SIGQUIT    4) SIGILL    5) SIGTRAP
 6) SIGABRT    7) SIGBUS    8) SIGFPE    9) SIGKILL   10) SIGUSR1
11) SIGSEGV   12) SIGUSR2   13) SIGPIPE   14) SIGALRM   15) SIGTERM
16) SIGSTKFLT   17) SIGCHLD   18) SIGCONT   19) SIGSTOP   20) SIGTSTP
21) SIGTTIN   22) SIGTTOU   23) SIGURG   24) SIGXCPU   25) SIGXFSZ
26) SIGVTALRM   27) SIGPROF   28) SIGWINCH   29) SIGIO   30) SIGPWR
```

Signaux : actions par défaut

Par défaut, la réception d'un signal entraîne :

- la mort du processus (par exemple `SIGTERM`, `SIGINT`, `SIGKILL`, ...) ou bien
- la mise en sommeil du processus (par exemple `SIGTSTP)`) ou bien
- le réveil du processus mis en sommeil (`SIGCONT`).

Certains signaux sont aussi simplement ignorés par défaut (par exemple `SIGCHLD`).

Cas d'utilisations

- Un processus cherche à lire (ou à écrire) dans une partie de la mémoire qui ne lui est pas attribuée. Le processeur émet une *interruption matérielle*, traitée par le noyau, qui envoie le signal `SIGSEGV` (pour *Segmentation Violation*) qui met fin au processus.
- Un processus parent rencontre une erreur irrécupérable. Le travail de ses enfants n'est plus pertinent, il les tue en leur envoyant un signal, par exemple `SIGINT`.

Envoyer un signal : `kill`

```
#include <signal.h>
int kill(pid_t pid, int sig);
```

- Si `pid > 0`, envoie le signal numéro `sig` (utiliser les constantes symboliques `SIGBIDULE`) au processus de pid `pid`.
- Si `pid = 0`, envoie le signal à tout le *process group* (hors-programme de ce cours).
- Si `pid = -1`, envoie le signal à tous les processus auxquels ce processus a le droit d'envoyer des signaux ! (sauf `1` et, sur Linux, lui-même).

Exemple (pédagogique)

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <signal.h>

int main(void)
{
    int i;
    pid_t p, autre;

    switch (p = fork()) {
    case -1: perror("fork"); return 1;
    case 0: autre = getppid(); break;
    default: autre = p;
    }

    printf("%d: Cette ville est trop petite pour nous deux, %d.\n",
          getpid(), autre);
    for (i = 3; i > 0; i--) {
        printf("%d\n", i);
        sleep(1);
    }
    kill(autre, SIGKILL);
    printf("Ouaouaou... Je suis %d et j'ai gagné mon duel.\n", getpid());
    return 0;
}
```

Situation (extrême) de compétition

- Tout peut se produire.
- L'un ou l'autre des processus peuvent gagner leur duel.
- Les processus peuvent même mourir tous les deux !

Autre exemple

```c
#include <signal.h>

int main(void)
{
    kill(-1, SIGTERM);
    return 0;
}
```

Vous pouvez vous reposer 1 minutes, le temps de relancer.

Statut de sortie et message d'erreur

Rappels de C : pointeurs (1)

Rappels de C, encore : fonctions avec paramètres pointeurs

Signaux : une introduction

Attendre : `wait`

Exécuter un programme : `exec`

Zombies

Synchronisation

```
#include <sys/types.h>
#include <sys/wait.h>
pid_t wait(int *wstatus);
```

- L'appel système `wait` permet pour un parent d'attendre la mort[2] d'un de ses enfants.
- `wait` est bloquant : si aucun enfant n'est mort (depuis le dernier appel à `wait`), le processus parent est *endormi* par le noyau et sera réveillé à la mort d'un enfant ;
- si un enfant est déjà mort (depuis le dernier appel à `wait`), le parent n'est pas endormi.
- argument `wstatus` : un pointeur qui permet d'écrire des informations sur la mort de l'enfant (de quoi est-il mort ?) ou `NULL` si on ne s'en préoccupe pas
- retourne le `PID` de l'enfant mort en question.
- S'il n'y a plus d'enfant à attendre, `wait` retourne -1.

2. En fait `wait` retourne aussi si un enfant a été endormi ou réveillé par un signal.

wait : exemple

```
int main(int argc, char *argv[])
{
    pid_t enf1, p;
    srand(time(NULL));
    int duree1 = rand() % 10000, duree2 = rand() % 10000;
    switch ((enf1 = fork())) {
    case -1: perror("fork"); return 1;
    case  0: usleep(duree1); return 0;
    default:
        switch (fork()) {
        case -1: perror("fork"); return 1;
        case  0: usleep(duree2); return 0;
        }
    }
    while ((p = wait(NULL)) != -1)
        if (p == enf1)
            printf("enfant 1 a terminé\n");
        else
            printf("enfant 2 a terminé\n");
    printf("Tout le monde a terminé.\n");
    return 0;
}
```

wait et statut de sortie

```
int main(int argc, char *argv[])
{
    pid_t enf1;
    int stat;
    switch ((enf1 = fork())) {
    case -1:
        perror("fork");
        return 1;
    case  0:
        srand(time(NULL));
        sleep(10);
        return rand() % 3;
    }
    printf("%d: j'ai un enfant qui a pour PID %d\n", getpid(), enf1);
    wait(&stat);
    if (WIFEXITED(stat))
        printf("il a terminé avec statut de sortie %d\n",
                WEXITSTATUS(stat));
    else
        printf("il a été tué par le signal %d\n", WTERMSIG(stat));
    return 0;
}
```

les macros `wait`

- `man 2 wait` (ne pas apprendre par cœur, mais savoir utiliser)
- `WIFEXITED(stat)` et `WIFSIGNALED(stat)` pour savoir si l'enfant est mort naturellement (`exit` ou `return` depuis `main`) ou a été tué par un signal
- `WEXITSTATUS(stat)` pour le statut de sortie dans le premier cas
- `WTERMSIG(stat)` pour le numéro du signal qui l'a tué dans le second cas
- (et encore deux derniers cas si le processus a été stoppé ou continué par un signal)

Statut de sortie et message d'erreur

Rappels de C : pointeurs (1)

Rappels de C, encore : fonctions avec paramètres pointeurs

Signaux : une introduction

Attendre : `wait`

Exécuter un programme : `exec`

Zombies

Exécuter un programme

- Le plus souvent on `fork` pour exécuter un nouveau programme, et non pour simplement créer une copie.
- On utilise dans ce cas, en général dans le processus enfant, un appel système de la famille `exec`
- qui remplace la pile, le tas, les données et le programme en chargeant en mémoire un nouveau programme.
- Redéfinit aussi les anciens arguments et peut, s'il le souhaite, redéfinir aussi l'environnement.

Un exemple

```
#include <unistd.h>
#include <stdio.h>
#include <sys/wait.h>

int main()
{
    switch (fork()) {
    case -1:
        perror("fork");
        return 1;
    case 0:
        execlp("ls", "ls", "-l", "/usr", NULL);
        perror("exec: ");
        return 1;
    }
    wait(NULL);
    printf("Commande ls terminée.\n");
    return 0;
}
```

Exemple (2)

```
$ gcc -Wall exemple_execlp.c
$ ./a.out
total 480
dr-xr-xr-x.   2 root root 122880 18 janv. 14:16 bin
drwxr-xr-x.   2 root root   4096 26 janv.  2021 games
drwxr-xr-x. 136 root root  12288 18 janv. 14:16 include
(...)
```

- Le programme du processus enfant a été entièrement remplacé par `ls` avec comme arguments `-l` et `/usr`.
- En particulier, la ligne `perror(...)` n'est atteinte qu'en cas d'erreur !
- exec ne retourne qu'en cas d'échec !
- La mémoire associée au processus enfant (pile, tas, zone de texte, zone de données) a aussi totalement changé.

La exec *family*

```
int execlp(const char *file, const char *arg0, const char *argv1,
           ..., NULL);
```

- `l` comme *list*, `p` comme `PATH`
- `file` : nom du fichier contenant l'exécutable. Pour cet appel, le `PATH` est utilisé pour la recherche de commandes si la chaîne d'adresse `file` ne contient pas de `/`.
- `arg0` : adresse d'une chaîne contenant ce qui sera `argv[0]`, par convention devrait être identique à `file`
- `arg1`, `arg2`, ... : les arguments
- toujours terminé par `NULL`.

La exec *family* (2)

```
int execvp(const char *file, char *const argv[]);
```

- `v` comme *vector*, `p` comme `PATH`.
- `file` : nom du fichier contenant l'exécutable. Pour cet appel, le `PATH` est utilisé pour la recherche de commandes si la chaîne d'adresse `file` ne contient pas de `/`.
- `argv` : tableau d'adresses de chaînes de caractère.
  - `argv[0]` : devrait être identique à `file` ;
  - `argv[1]` : premier argument
  - ...
  - `argv[argc]` : doit être `NULL`, signale la fin du tableau.

La `exec` *family* (3)

- `l` comme *list*, `v` comme *vector*, `e` comme *environment* et `p` comme `PATH`.
- `execl`, comme `execlp` mais sans `PATH`
- `execv`, comme `execvp` mais sans `PATH`
- `execle`, comme `execl` mais en spécifiant l'environnement
- `execvpe`, comme `execvp` mais en spécifiant l'environnement
- `man 3 exec`

Que peut-on `exec` ?

- D'abord il faut que le processus (donc en général l'utilisateur qui l'a lancé) ait la permission d'exécuter (`x`) le fichier ;
- ensuite il faut que le noyau puisse comprendre de quoi il s'agit...
- le fichier doit donc être soit dans un format binaire compilé bien spécifique (sur Linux, Elf64 ou Elf32, autre chose sous MacOS, ce sont les compilateurs qui sont responsables de produire de tels fichiers),...
- ou bien un fichier commençant par les deux octets `#!`, suivi d'un chemin `/chemin/vers/programme`, dans ce cas le noyau remplace le programme du processus par `/chemin/vers/programme` en lui donnant le fichier à interpréter (à lire).

Exemple : `exec` un script shell

Supposons qu'un fichier `plop` est exécutable

```
$ ls -l plop
-rwxr-xr-x. 1 pierre pierre 75  3 févr. 10:49 plop
```

Et contient le script shell suivant :

```
#!/bin/sh
i=0
while [ "$i" -lt 4 ]; do
        printf 'lol\n'
        i=$((i + 1))
done
```

Exécuter un script

Dans un programme `exec_script.c`, situé dans le même répertoire :

```
int main(void)
{
    switch (fork()) {
    case -1: /* gérer l'erreur */
    case 0:
        execl("plop", "plop", NULL);
        perror("exec: ");
        return 1;
    }
    wait(NULL);
    printf("Commande plop terminée.\n");
    return 0;
}
```

On obtient le résultat suivant :

```
$ gcc -Wall exec_script.c -o plop_depuis_c
$ ./plop_depuis_c
lol
lol
lol
lol
Commande plop terminée.
```

Statut de sortie et message d'erreur

Rappels de C : pointeurs (1)

Rappels de C, encore : fonctions avec paramètres pointeurs

Signaux : une introduction

Attendre : `wait`

Exécuter un programme : `exec`

Zombies

Zombies

- Le statut de sortie d'un processus est une information qui peut être très importante.
- Lorsqu'un processus meurt, mais que son parent est vivant et
  - ne l'a pas encore `wait` ou
  - ne compte jamais le faire (mais ça, personne ne peut le savoir...)

  le noyau conserve le processus mort dans sa table de processus (mais le processus lui-même ne prend plus aucune ressource).

Zombies

- Le statut de sortie d'un processus est une information qui peut être très importante.
- Lorsqu'un processus meurt, mais que son parent est vivant et
  - ne l'a pas encore `wait` ou
  - ne compte jamais le faire (mais ça, personne ne peut le savoir...)

  le noyau conserve le processus mort dans sa table de processus (mais le processus lui-même ne prend plus aucune ressource). Ce processus est donc mort, mais encore dans la table, c'est un zombie.

```c
#define NB_Z_DFT 5
int main(int argc, char *argv[]) {
    int nb_z = argc > 1 ? atoi(argv[1]) : NB_Z_DFT;
    for (; nb_z > 0; --nb_z)
        switch (fork()) {
        case -1: perror("fork"); return 1;
        case 0:
            printf("%d: Brains!\n", getpid());
            if (nb_z % 3 == 0) /* se tue avec SIGTERM */
                kill(getpid(), SIGTERM);
            else if (nb_z % 3 == 1) /* se tue avec SIGKILL */
                kill(getpid(), SIGKILL);
            else
                return nb_z;
        }
    sleep(10); /* regarder ps -l */
    pid_t p;
    int stat;
    while ((p = wait(&stat)) != -1) {
        if (WIFEXITED(stat))
            printf("%d est mort normalement avec statut %d.\n",
                p, WEXITSTATUS(stat));
        else
            printf("%d a été tué par le signal numéro %d.\n",
                p, WTERMSIG(stat));
    }
    /* Il n'y a plus de zombies ! */
    for (;;) sleep(120); /* dort pour toujours */
}
```

Même chose pour ceux qui aiment les scripts shell

```
#!/bin/sh
NB_Z_DFT=5
nb_z=${1:-$NB_Z_DFT}
while [ $nb_z -gt 0 ]; do
        sleep 2 &
        nb_z=$(($nb_z - 1))
done
exec sleep 200
```

Même chose pour ceux qui aiment les scripts shell

```
#!/bin/sh
NB_Z_DFT=5
nb_z=${1:-$NB_Z_DFT}
while [ $nb_z -gt 0 ]; do
        sleep 2 &
        nb_z=$(($nb_z - 1))
done
exec sleep 200
```

Comprenez-vous comment ce script crée des zombies ?